

Binary Codes

In the coding, when **numbers, letters or words are represented** by a **specific group of symbols**, it is said that the number, letter or word is being **encoded**. The **group of symbols is called as a code**. The **digital data is represented, stored and transmitted as group of binary bits**. This group is also called as **binary code**. The binary code is represented by the number as well as alphanumeric letter.

Advantages of Binary Code

Following is the list of advantages that binary code offers:

- Binary codes are **suitable for the computer applications**.
- Binary codes are **suitable for the digital communications**.
- Binary codes make the **analysis and designing of digital circuits much easier**.
- Since only 0 & 1 are being used, **implementation becomes easy**.

Classification of Binary Codes

The binary codes are categorized into following categories:

- Binary Coded Decimal (BCD) Code
- Gray Code
- Excess-3 Code
- Alphanumeric Codes
- Error Detecting Codes
- Error Correcting Codes

Binary Coded Decimal (BCD)

In this code each **decimal digit is represented by a 4-bit binary number**. BCD is a way to express each of the decimal digits with a binary code. In the BCD, with **four bits** we can **represent sixteen numbers (0000 to 1111)**. But in BCD code **only first ten (0000 to 1001)** of these are **used**. The **remaining six** code combinations i.e., 1010 to 1111 are **invalid** in BCD.

Decimal	0	1	2	3	4	5	6	7	8	9
BCD	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001

Advantages of BCD Codes

- It is very **similar to decimal** system.
- We **need to remember binary equivalent** of decimal numbers 0 to 9 only.

Disadvantages of BCD Codes

- The **addition and subtraction** of BCD have different rules.
- The BCD **arithmetic** is little more complicated.
- BCD **needs a greater number of bits** than binary to **represent the decimal** number. So, BCD is **less efficient** than binary.

Gray Code

- The **Gray Code** is a sequence of binary number systems, which is also known as **Reflected Binary Code (RBC)**. The reason for calling this code as reflected binary code is the **first $N/2$ values compared with those of the last $N/2$ values in reverse order**.
- Gray codes are **used** in the general **sequence of hardware-generated binary numbers**. These numbers **cause ambiguities or errors** when the **transition from one number to its successive** is done. The **gray code simply solves this problem by changing only one bit** when the **transition is between numbers** is done. Changing only one bit **reduces the switching operation** also.
- Gray code is an **unweighted code**. The Gray code is a very light weighted code because it **doesn't depend on the value of the digit specified by the position**.
- This code is also called a **cyclic variable code** as the **transition** of one value to its successive value **carries a change of one bit** only.

Therefore, the **two most important characteristics** of Gray code are:

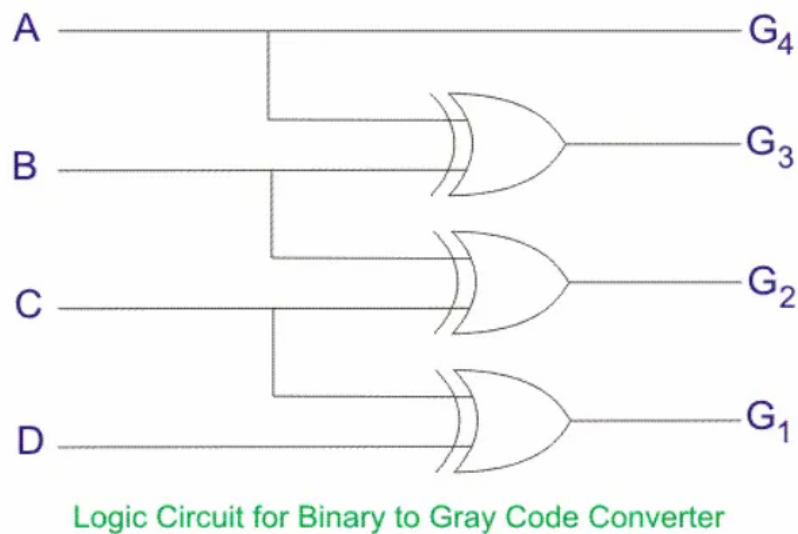
- In Gray code, **two consecutive values are differed by one bit** of binary digits
- Binary number is **converted to Gray code to reduce the switching operation**.

Binary to Gray Code Conversion

- In the Gray code, the **MSB will always be the same** as the 1'st bit of the given binary number.
- In order to perform the **2nd bit of the gray code**, we **perform the exclusive-or (XOR) of the 1'st and 2nd bit** of the binary number. It means that if **both the bits are different**, the result will be **one** else the result will be **0**.
- In order to get the **3rd bit of the gray code**, we need to perform the **exclusive-or (XOR) of the 2nd and 3rd bit** of the binary number.
- The **process remains the same** for the 4th bit of the Gray code.

Logic Circuit for Binary to Gray Code Converter

The binary to gray code converter circuit is shown below:

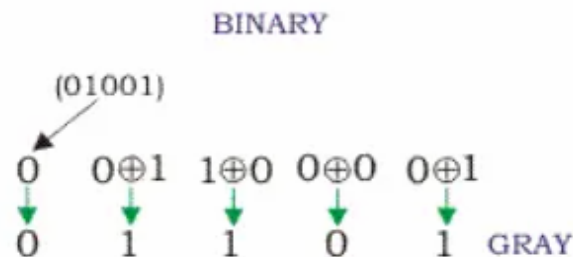


Binary to Gray Code Conversion Example

Say we have a **binary number 01001** which we wish to convert to gray code. Let's go through an example of how we would perform this conversion:

- The MSB is kept the same. As the MSB of the binary is 0, the MSB of the gray code will be 0 as well (first gray bit)

- Next, take the XOR of the first and the second binary bit. The first bit is 0, and the second bit is 1. The bits are different so the resultant gray bit will be 1 (second gray bit)
- Next, take the XOR of the second and third binary bit. The second bit is 1, and the third bit is 0. These bits are again different so the resultant gray bit will be 1 (third gray bit)
- Next, take the XOR of third and fourth binary bit. The third bit is 0, and the fourth bit is 0. As these are the same, the resultant gray bit will be 0 (fourth gray bit)
- Lastly, take the XOR of the fourth and fifth binary bit. The fourth bit is 0, and the fifth bit is 1. These bits are different so the resultant gray bit will be 1 (fifth gray bit)
- Hence the result of binary to gray code conversion of 01001 is complete, and the equivalent gray code is 01101.



Gray to Binary Code Conversion

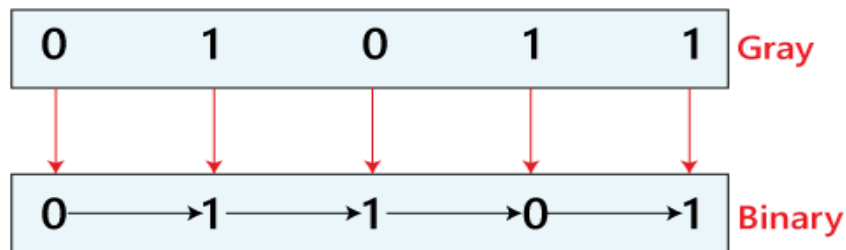
Just like Binary to Gray code conversion; it is also a very simple process. There are the following steps used to convert the Gray code into binary:

- The **MSB** of the binary number **will be equal** to the MSB of the given gray code.
- Now if the **second gray bit is 0**, then the **second binary bit will be the same as the previous** or the first bit. If the **gray bit is 1** the **second binary bit will alter**. If it was 1 it will be 0 and if it was 0 it will be 1.
- This **step is continued** for all the bits to do Gray code to binary conversion.

Gray Code to Binary Conversion Example

Suppose we have the **Gray code 01011**, which we want to convert into a binary number. There are the following steps which we need to perform for the conversion:

- The 1st bit of the binary number is the same as the MSB of the Gray code. The MSB of the Gray code is 0, so the MSB of the binary number is 0.
- Now, for the 2nd bit, we check the 2nd bit of the Gray code. The 2nd bit of the Gray code is 1, so the 2nd bit of the binary number is one that is altered number of 1st
- The next bit of the Gray code is 0; the 3rd bit is the same as the 2nd bit of the Gray code, i.e., 1.
- The 4th bit of the Gray code is 1; the 4th bit of the binary number is 0 that is the altered number of the 3rd
- The 5th bit of the Gray code is 1; the 5th bit of the binary number is 1; that is the altered number of the 4th bit of the binary number.
- So, the binary number of the Gray code 01011 is 01101.



Gray Code Table

Decimal Number	Binary Number	Gray Code
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100

8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

Excess-3 Code

- The excess-3 code is also treated as **XS-3 code**.
- The excess-3 code is a **non-weighted and self-complementary BCD code** used to represent the decimal numbers.

Methods for Finding Excess-3 Code of a Binary Number

We can **find the excess-3** code of the given binary number by using the **following two methods**:

Method 01:

- We **find the decimal number** of the given binary number.
- Then we **add 3 in each digit** of the decimal number.
- Now, we **find the binary code of each digit** of the newly generated decimal number.

Method 02:

We can also **add 0011 in each 4-bit BCD code** of the decimal number for getting excess-3 code.

The Excess-3 code for the decimal number is as follows:

Decimal Digit	BCD Code	Excess-3 Code
0	0000	0011
1	0001	0100
2	0010	0101
3	0011	0110
4	0100	0111
5	0101	1000
6	0110	1001
7	0111	1010
8	1000	1011
9	1001	1100

In excess-3 code, the codes **1111** and **0000** are never used for any decimal digit.

Examples to Find Excess-3 Code

Let's take some examples of Excess-3 code.

Example 1:Decimal number 31

Step 1: We find the **BCD code of each digit** of the decimal number.

Digit	BCD
3	0011
1	0001

Step 2: Then, we **add 0011** in both of the BCD code.

Decimal	BCD	Excess-3
---------	-----	----------

3	0011+0011	0110
1	0001+0011	0100

Step 3: So, the **excess-3 code** of the decimal number 31 is **0110 0100**

Example 2:Decimal number 81.61

Step 1: We find the **BCD code of each digit** of the decimal number.

Digit	BCD
8	1000
1	0001
6	0110
1	0001

Step 2: Then, we **add 0011** in both of the BCD code.

Decimal	BCD	Excess-3
8	1000+0011	1011
1	0001+0011	0100
6	0110+0011	1001

Step 3: So, the **excess-3 code** of the decimal number 81.61 is:
1011 0100.1001 0100

Self-Complementary Property

- A self-complementary binary code is a code which is **always complimented in itself**. By **replacing the bit 0 to 1 and 1 to 0** of a number, we find the **1's complement** of the number.

- If we perform the **1's complement of excess-3** of a decimal number, it will be **equal** to the **excess-3** code of the **9's complement** of that decimal number.

9's Complement	Decimal Digit	BCD Code	Excess-3 Code
9	0	0000	0011
8	1	0001	0100
7	2	0010	0101
6	3	0011	0110
5	4	0100	0111
4	5	0101	1000
3	6	0110	1001
2	7	0111	1010
1	8	1000	1011
0	9	1001	1100

For example: Given, Decimal number is **5**. It's **excess-3** code is **1000**. If we perform **1's complement**, the complemented value will be **0111**. This is the **excess-3** code of **9's complement** of **5**. Therefore, **they are equal**.

Alphanumeric Codes

A **binary digit** or **bit** can represent **only two symbols** as it has only two states '**0**' or '**1**'. But this is **not enough for communication** between two computers because there we **need many more symbols** for communication. These symbols are **required** to represent **26 alphabets** with **capital and small** letters, **numbers from 0 to 9**, **punctuation marks** and **other symbols**.

We know that a **computer understands only binary language** (0 and 1). Moreover, it is **not able to directly understand or store any alphabets, other numbers, pictures, symbols, etc.** Therefore, we **use certain coding schemes** so that it can understand each of them correctly. Besides, we call **these codes alphanumeric codes**.

- The **alphanumeric codes** are the codes that **represent numbers and alphabetic characters**.
- Mostly such codes **also represent** other characters such as **symbol and various instructions** necessary for **conveying information**.
- An alphanumeric code **should at least represent 10 digits and 26 letters** of alphabet i.e., **total 36 items**.

The **following three alphanumeric codes** are very commonly used for the data representation.

- American Standard Code for Information Interchange (ASCII).
- Extended Binary Coded Decimal Interchange Code (EBCDIC).
- Unicode.

American Standard Code for Information Interchange (ASCII)

- ASCII is a code for **representing 128 English characters** as numbers, with each letter assigned a number **from 0 to 127**. For example, the **ASCII code for uppercase M is 77**.
- Most computers use **ASCII codes** to **represent text**, which **makes it possible to transfer data** from one computer to another.
- The **standard ASCII** character set uses just **7 bits** for each character.
- There are several **larger character sets that use 8 bits**, which gives them **128 additional characters**. The **extra characters** are used to **represent non-English characters, graphics symbols, and mathematical symbols**.

ASCII vs EBCDIC

- **ASCII code is a 7-bit code** whereas **EBCDIC is an 8-bit code**.
- **ASCII code is more commonly used** worldwide while **EBCDIC is used primarily in large IBM computers**.
- **EBCDIC contained more characters** than ASCII.
- **ASCII uses a linear ordering** of letters while **EBCDIC does not**.

- **Different versions of ASCII are mostly compatible** while different versions of **EBCDIC are not.**
- **EBCDIC isn't compatible with modern encodings** while **ASCII is.**

UNICODE (Multilingual Computing)

Unicode is a standard for **character encoding**. The introduction of **ASCII characters was not enough to cover all the languages**. Therefore, to overcome this situation, it was introduced. The **Unicode Consortium** introduced this encoding scheme.

Unicode is a **universal character encoding standard**. This standard includes roughly **100000 characters to represent characters of different languages**. While **ASCII uses only 1 byte** the **Unicode uses 4 bytes to represent characters**. Hence, it provides a very wide variety of encoding. It has **three types namely UTF-8, UTF-16, UTF-32**. Among them, **UTF-8 is used mostly** it is also **the default encoding for many programming languages**.

UCS

It is a very common acronym in the Unicode scheme. It stands for **Universal Character Set**. Furthermore, it is the encoding scheme for storing the Unicode text.

- **UCS-2:** It uses two bytes to store the characters.
- **UCS-4:** It uses four bytes to store the characters.

UTF

The UTF is the most important part of this encoding scheme. It stands for **Unicode Transformation Format**. Moreover, this defines how the code represents Unicode. It has **4 types** as follows:

UTF-7

This scheme is **designed to represent the ASCII standard**. Since the **ASCII uses 7 bits encoding**. It **represents the ASCII characters in emails and messages** which use this standard.

UTF-8

It is the **most commonly used form** of encoding. Furthermore, it **has the capacity to use up to 4 bytes for representing the characters**. It uses:

- 1 byte to represent English letters and symbols.
- 2 bytes to represent additional Latin and Middle Eastern letters and symbols.
- 3 bytes to represent Asian letters and symbols.
- 4 bytes for other additional characters.

Moreover, it is **compatible with the ASCII standard**.

Its **uses are as follows**:

- Many protocols use this scheme.
- It is the default standard for XML files
- Some file systems Unix and Linux use it in some files.
- Internal processing of some applications.
- It is widely used in web development today.
- It can also represent emojis which is today a very important feature of most apps.

UTF-16

It is an **extension of UCS-2 encoding**. Moreover, it uses to **represent the 65536 characters**. Moreover, it **also supports 4 bytes for additional characters**. Furthermore, it is **used for internal processing like in java, Microsoft windows, etc**.

UTF-32

It is a **multibyte encoding scheme**. Besides, it **uses 4 bytes to represent the characters**.

Importance of Unicode

- As it is a universal standard therefore, it **allows writing a single application for various platforms**. This means that we can **develop an application once and run it on various platforms in different languages**. Hence, we don't have to write the code for the same application again and again. Therefore, the **development cost reduces**.
- Moreover, **data corruption is not possible** in it.
- It is a **common encoding standard** for many different languages and characters.
- We can **use it to convert from one coding scheme to another**. Since **Unicode is the superset for all encoding schemes**. Hence, we can convert a code into Unicode and then convert it into another coding standard.
- It is **preferred by many coding languages**. For example, XML tools and applications use this standard only.

Difference Between Unicode and ASCII

The differences between them are as follows:

Unicode Coding Scheme	ASCII Coding Scheme
It uses variable bit encoding according to the requirement. For example, UTF-8, UTF-16, UTF-32	It uses 7-bit encoding. As of now, the extended form uses 8-bit encoding.
It is a standard form.	It is not a standard all over the world.
People use this scheme all over the	It has only limited characters hence, it cannot

world.	be used all over the world.
The Unicode characters themselves involve all the characters of the ASCII encoding. Therefore, we can say that it is a superset for it.	It has its equivalent coding characters in the Unicode.
It has more than 128,000 characters.	In contrast, it has only 256 characters.

Error Detection and Correction Code

- In the **transmission of data** from one source to another, **noise gets added** into the data.
- This **causes errors** in the received binary data at the receiver systems.
- The **bits of the data** may **change (either 0 to 1 or 1 to 0)** during transmission.
- It is **impossible to avoid the interference of noise**.
- But it is **possible to get back the original data**.
- For this purpose, we **first need to detect** either an **error is present or not** using error detection codes.
- If the **error is present** in the code, then we will **correct it with the help of error correction codes**.

